Brain Tumor Classifier

Pattern Recognition Systems - A25

LEDDA Romain

What is the problem ?

A brain tumor can be benign or malignant. The earlier the diagnosis is made, the earlier you can treat the patient. Or, manual MRI analysis (Magnetic Resonance Imaging) is time-consuming and prone to human error.

Objective : Automate the detection process using Machine Learning algorithms like KNN or SVM.

Related Work

- M. S. I. Khan *et al.*, « Accurate brain tumor detection using deep convolutional neural network », *Computational and Structural Biotechnology Journal*, vol. 20, p. 4733-4745, janv. 2022, doi: **10.1016/j.csbj.2022.08.039**

This research paper introduces a 23-layer deep Convolutional Neural Network (CNN) that eliminates the need for manual feature extraction. They achieve near-perfect accuracy (between 97% and 100%) on standard benchmark datasets.

In the end, I preferred to improve my results for my SVM classifier rather than implement this architecture.

- The implementation of KNN Classifier in C++, we did in lab classes
- The documentation of scikit-learn for the SVM classifier implementation

Proposed Methods

K-Nearest Neighbors (KNN)

Concept: KNN is a classifier that assigns a label to a test image based on the most common class among its closest neighbors in the feature space.

Mechanism: It stores all training samples and, for each new image, calculates the **Euclidean distance** between its color or texture histogram and the training histograms. The algorithm selects the "K" nearest neighbors and performs a majority vote to decide if the scan shows a tumor or not.

Pros & Cons : It is a simple classifier but it can be computationally expensive as it grows with the size of the dataset.

Support Vector Machine (SVM)

Concept: SVM is a classifier designed to find the optimal boundary, called a **hyperplane,** that separates two classes (No tumor vs. Tumor) with the maximum possible margin.

Mechanism: During training, it identifies the most critical data points, which are called **Support Vectors**, that define this boundary. It can use "Kernels" (like RBF or Linear) to transform complex data, making it easier to separate classes that are not linearly separable in their raw state.

Pros & Cons : Once the hyperplane is found, prediction is nearly instantaneous but it takes a long time to train.

KNN Classifier

Input

- K: Number of neighbors (Ex : K=4)
- X_train : Matrix of (p x n) histograms from the training set with p the the number of images, and n, the number of features. Here, the colors.
- y_train: Known classes for each image (0: No Tumor, 1: Tumor)
- X_test : Histogram of the image to classify
- y_test : Known classes or labels of the test images

Output

- Predicted Class Label ("Tumor", "No Tumor")

KNN Classifier

Algorithm

1. For each entry in X_train :
    a. Calculate the Euclidean Distance between X_test and X_test(i)
    b. Store the (distance, label) pair in a list
2. Sort the list by distance in ascending order (closest first)
3. Select the first K elements from the sorted list (the K-nearest neighbors)
4. Count the occurrences of each class label among these K neighbors
5. Assign the X_test to the class with the highest frequency (Majority Vote)

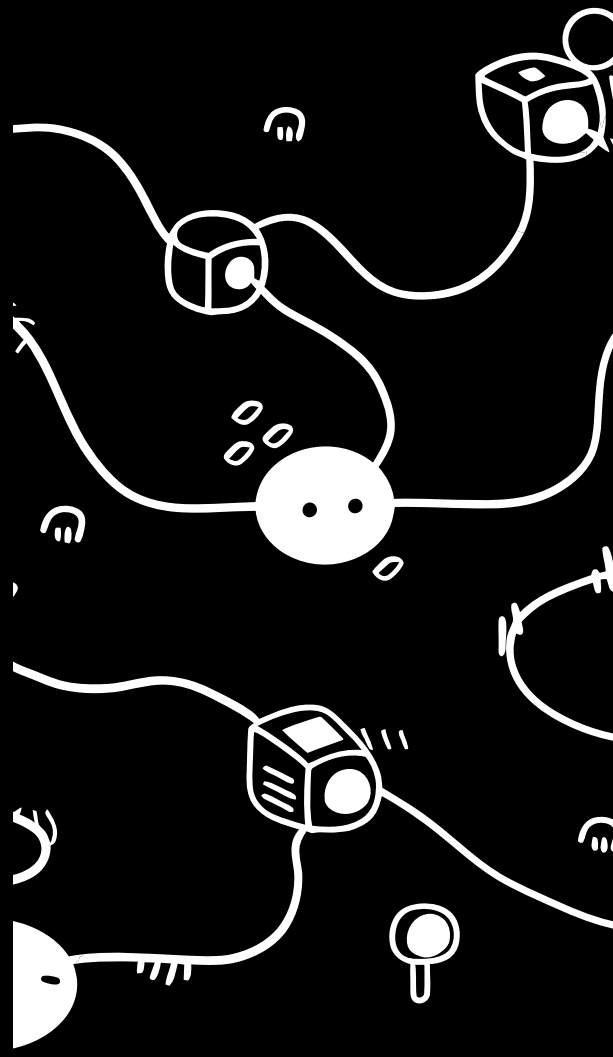
SVM Classifier

Input

- X_train : Matrix of (p x n) histograms from the training set with p the the number of images, and n, the number of features. Here, the LBP features.
- y_train: Known classes for each image (0: No Tumor, 1: Tumor)
- X_test : Histogram of the image to classify
- y_test : Known classes or labels of the test images
- Kernel : RBF (Radial Basis Function) or Linear
- C : Regularization parameter

Output

- Predicted Class Label



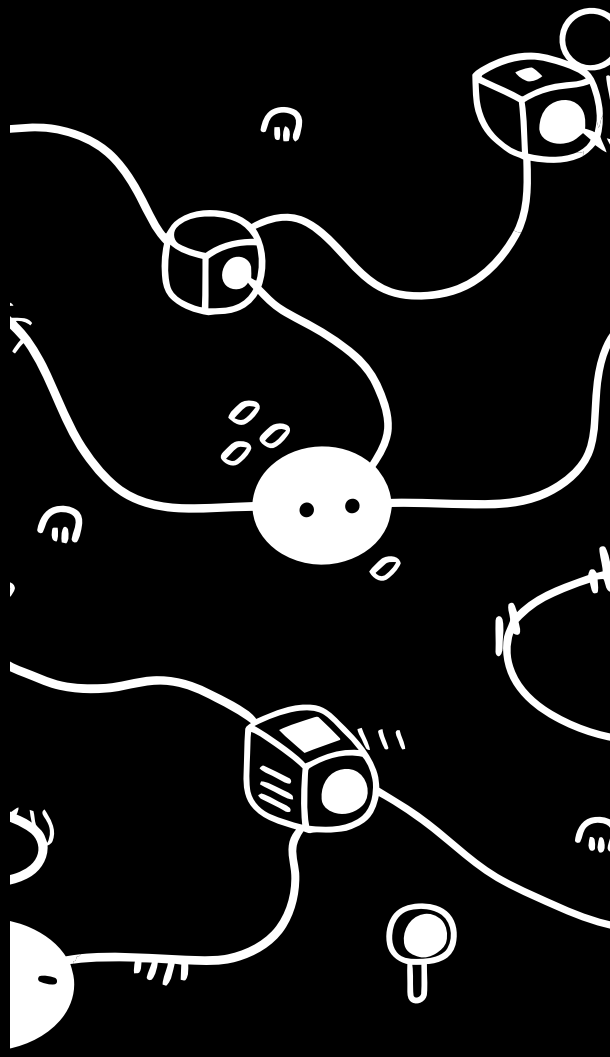
SVM Classifier

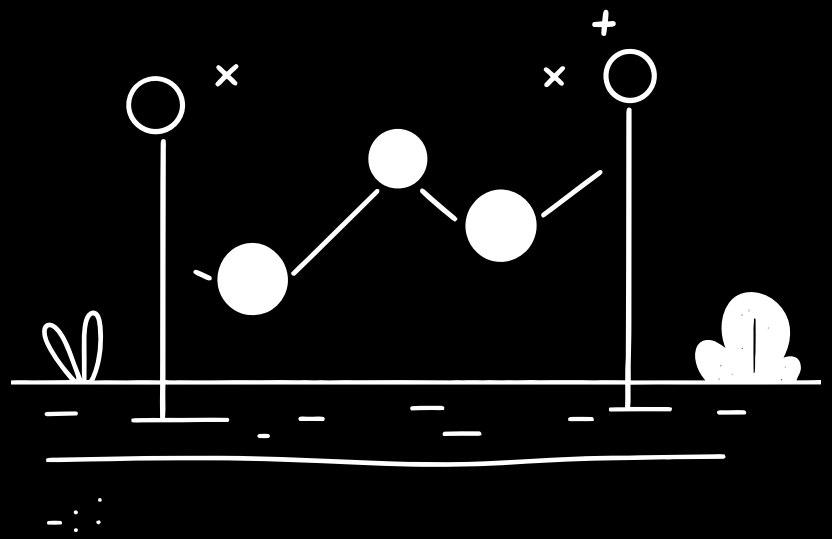
Algorithm Training Phase

1. Map the LBP features into a high-dimensional space
2. Solve an optimization problem to find a decision boundary (hyperplane) :
    - Maximize the margin between the two classes (No Tumor vs. Tumor)
    - Minimize classification errors on the training set (weighted by C)
3. Identify "Support Vectors," which are the data points closest to the hyperplane
4. Store the weights (W) and bias (b) defining the hyperplane

Algorithm Prediction Phase

1. Input a new X_feature x
2. Compute the decision function: $f(x) = \text{sign}(W \cdot x + b)$
3. If $f(x) > 0$, classify as Tumor; else, classify as No Tumor





Implementation details

KNN

Developped functions

- save_knn_data(X_train, y_train, X_test, y_test, filename)
- load_knn_data(filename)
- load_data(data_dir, classes, train_step)
- calculate_LBP_histogram(img, m=256)
- calculate_histogram(img, m)
- predict_knn(k, X_train, y_train, hist_test)
- evaluate_knn(dir_path, categories, K = 3)
- advanced_metrics(C)

Libraries used

- kagglehub, cv2, os, numpy, tqdm
- local_binary_pattern() from skimage

Code

```python
def load_data(data_dir, classes, train_step):
    m = 256
    # X : ce sont les images
    X_features = []
    y_labels = []
    # y : Ce sont les labels

    print(f"\n---------------- Chargement des données {train_step} -------------")

    for class_name in classes:
        label_id = classes.index(class_name)
        path = os.path.join(data_dir, train_step, class_name)

        if not os.path.isdir(path):
            print(f"[ATTENTION] : le dossier {path} n'existe pas")
            return

        file_list = [f for f in os.listdir(path) if f.lower().endswith('.jpg')]

        for filename in tqdm(file_list, desc=f"Traitement {class_name}"):
            img_path = os.path.join(path, filename)

            img = cv2.imread(img_path, cv2.IMREAD_COLOR)

            if img is not None:
                # hist_vector = calculate_histogram(img, m)
                hist_vector = calculate_LBP_histogram(img, m)
                X_features.append(hist_vector)
                y_labels.append(label_id)

    return np.array(X_features), np.array(y_labels)
```

```python
def calculate_LBP_histogram(img, m=256):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    radius = 1
    n = 8 * radius
    lbp = local_binary_pattern(gray, n, radius, method="uniform")
    n_bins = int(lbp.max() + 1)
    hist, _ = np.histogram(lbp.ravel(), bins=n_bins, range=(0, n_bins), density=True)
    hist = hist.astype("float32")
    hist /= (hist.sum() + 1e-7)
    return hist

def calculate_histogram(img, m):
    img = cv2.resize(img, (128, 128)) # Amélioration

    hist = np.zeros(3 * m)

    if img is None :
        return np.zeros(m * 3, dtype=np.float32)

    height, width, _ = img.shape

    for row in range(height):
        for col in range(width):
            pixel = img[row, col]
            hist[pixel[0]] += 1
            hist[int(pixel[1]) + m] += 1
            hist[int(pixel[2]) + (2 * m)] += 1

    size = height * width

    if size > 0:
        hist /= size

    return hist
```

```python
def predict_knn(k, X_train, y_train, hist_test):

    # distance_array = np.sqrt(np.sum((X_train - hist_test)**2, axis=1))

    list_distance = []
    num_train_examples = X_train.shape[0]

    for row in range(num_train_examples):
        distance = 0.0
        hist_train = X_train[row]

        for col in range(hist_train.shape[0]):
            distance += (hist_train[col] - hist_test[col]) ** 2;
        distance = sqrt(distance)
        list_distance.append((distance, y_train[row]))

    list_distance.sort(key = lambda x : x[0])

    k_nearest_neighbors = list_distance[:k]
    # k_nearest_neighbors = np.argsort(distance_array)[:k]

    classes_occurence = {}

    for _, label in k_nearest_neighbors:
        classes_occurence[label] = classes_occurence.get(label, 1) + 1

    predicted_class = max(classes_occurence.items(), key=lambda item : item[1])[0]

    # neighbor_labels = y_train[k_nearest_neighbors]
    # predicted_class = np.argmax(np.bincount(neighbor_labels))

    return predicted_class
```

```python
def evaluate_knn(dir_path, categories, K = 3):

    data = load_knn_data()
    if data is None:
        X_train, y_train = load_data(dir_path, categories, train_step="Training")
        X_test, y_test = load_data(dir_path, categories, train_step="Testing")
        save_knn_data(X_train, y_train, X_test, y_test)
    else:
        X_train, y_train, X_test, y_test = data

    num_classes = len(categories)
    num_test_examples = X_test.shape[0]

    C = np.zeros((num_classes, num_classes))

    print(f"\n--- Évaluation des {num_test_examples} échantillons de test (K={K}) ---")

    for row in tqdm(range(num_test_examples), desc="Prédiction des tests"):
        c_ground_truth = y_test[row]
        hist_test = X_test[row]
        c_predicted = predict_knn(K, X_train, y_train, hist_test)
        C[c_predicted, c_ground_truth] += 1

    correct_predictions = np.trace(C)
    total_predictions = np.sum(C)

    accuracy = correct_predictions / total_predictions

    return accuracy, C
```

SVM

Developped functions

- save_svm_model(model, filename)
- load_svm_model(filename)
- train_svm(X_train, y_train, kernel, C)
- evaluate_svm(model, X_test, y_test)

Libraries used

- kaglehub, numpy, os, kagglehub, math; tqdm, joblib
- functions from knn module
- SVC, confusion_matrix, accuracy_score from sklearn

Code

```python
def train_svm(X_train, y_train, kernel="rbf", C=1.0):
    print(f"-" * 20 + "Entrainement du SVM avec Kernel : {kernel}" + "-" * 20)
    model = SVC(kernel=kernel, C=C, class_weight="balanced")
    model.fit(X_train, y_train)
    save_svm_model(model, "medical_svm")
    print(f"-"*20 + "Entrainement du SVM terminé" + "-"*20)
    return model

def evaluate_svm(model, X_test, y_test):
    print(f"-" * 20 + "Evaluation du SVM" + "-" * 20)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    C = confusion_matrix(y_test, y_pred)

    print(f"-" * 20 + "Evaluation du SVM terminée" + "-" * 20)      You, last mo

    C = C.T

    return accuracy, C
```

```python
# class_names = ["notumor", "glioma"]
class_names = ["no_tumor", "glioma_tumor"] # Dataset 2

# data_path = kagglehub.dataset_download("masoudnickparvar/brain-tumor-mri-dataset") #
data_path = kagglehub.dataset_download("sartajbhuvaji/brain-tumor-classification-mri")
print("Path to dataset files:", data_path)

print("\n" + "-"*60)
print(f" SVM Classifier ".center(60))
print("=" * 60)

# data = load_knn_data("knn_cache.npz")
data = load_knn_data("knn_cache_lbp_dataset_2.npz")
if data is None:
    X_train, y_train = load_data(data_path, class_names, train_step="Training")
    X_test, y_test = load_data(data_path, class_names, train_step="Testing")
    save_knn_data(X_train, y_train, X_test, y_test)
else:
    X_train, y_train, X_test, y_test = data

svm_model = load_svm_model("medical_svm")
if svm_model is None:
    svm_model = train_svm(X_train, y_train)

accuracy, C = evaluate_svm(svm_model, X_test, y_test)
recall, precision, f1_score = advanced_metrics(C)
correct_samples = np.trace(C)
total_samples = np.sum(C)

print("="*60)
print("  RÉSUMÉ DES RÉSULTATS  ".center(60))
print("="*60)
print(f"| Accuracy : {accuracy:.4f} ({accuracy*100:.2f} %)".ljust(59) + "|")
```

Results - Dataset 1

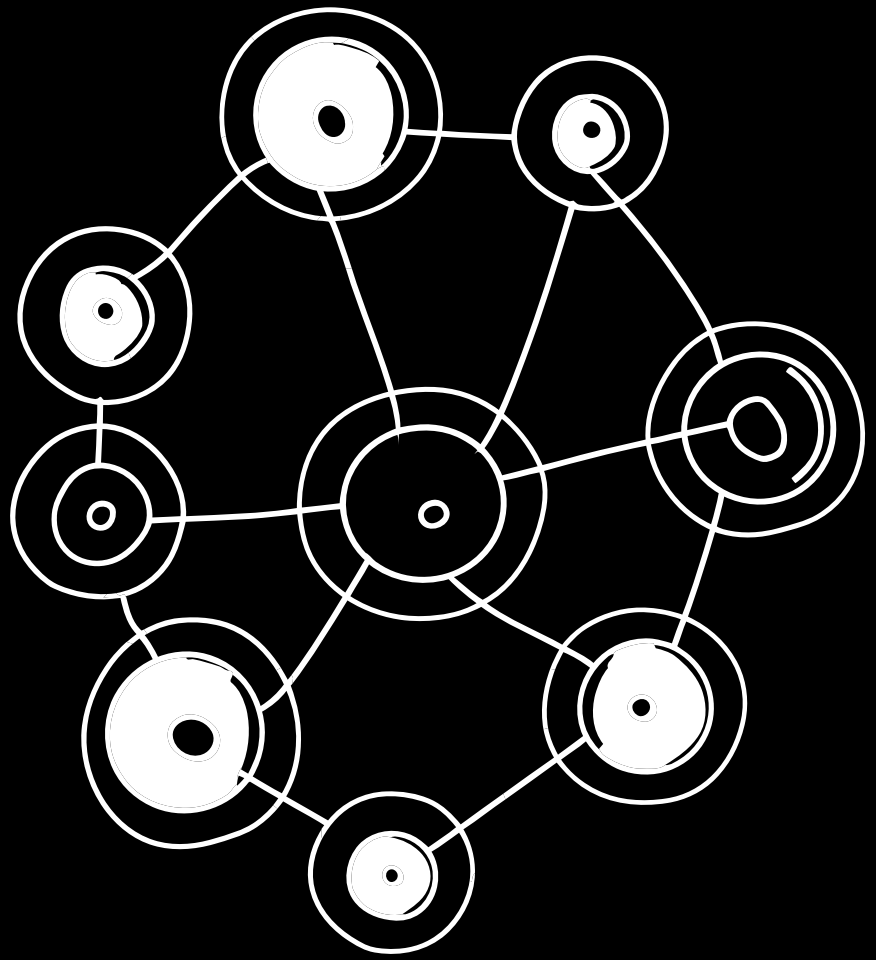
	KNN (k = 3)	SVM
Accuracy	99.29 %	99.15 %
Recall	100.0 %	99.0 %
Precision	98.36 %	99.0 %
F1-Score	99.17 %	99.0 %
Correct Precisions	700/705	699/705

Results - Dataset 2 - Color Histogram

	KNN (k = 3)	KNN (k = 4)	SVM
Accuracy	61.46 %	62.44 %	57.56 %
Recall	**25.00 %**	27.00 %	20.00 %
Precision	86.21 %	87.10 %	74.07 %
F1-Score	38.76 %	41.22 %	31.50 %
Correct Precisions	126/205	128/205	118/205

Results - Dataset 2 - LBP

	KNN (k = 3)	SVM
Accuracy	69.79 %	70.24 %
Recall	**40.00 %**	43.00 %
Precision	95.24 %	91.49 %
F1-Score	56.34 %	58.50 %
Correct Precisions	143/205	144/205



Conclusion

What Works: We have good results for ML techniques. Transitioning from pixel intensity to LBP provided a significant performance boost, raising accuracy from 57.56% to 70.24%. It is equivalent for KNN.

Limitations

- **Low Recall (~40%)** : Despite high precision, the model fails to detect ~60% of the tumors. This suggests that texture alone is insufficient for a comprehensive medical diagnosis.
- LBP or pixel intensity cannot capture complex spatial hierarchies, links or abstract patterns in MRI scanner.

Future Work: We need the **23-layer CNN** from Khan et al., which uses automated feature extraction to improve our Recall. This will allow to learn more complex characteristics.